MT-1008 Multivariable Calculus

BS CS Spring 2026

Final Project

(Group Members)

Name:	Roll Number:
Qais Mubeen (Group Leader)	25I-0657
Mobeen Ahmed	25I- 0972
Junaid Asif	25I-0852
Section:	CS-H
Submitted To:	Mr. Huzam
Submission Date:	02-05-2026

Multivariable Calculus Project:
Traffic Flow Dynamics: Multivariable Modeling and Analysis

1. Introduction

Traffic congestion is one of those stubborn problems that urban planners just can't seem to fully resolve. That said, when we bring mathematics into the picture, we can actually model and predict how vehicles behave on roads, laying the groundwork for far smarter transportation systems. This project takes that idea seriously and applies multivariable calculus to study traffic flow dynamics, drawing on a 30-day dataset gathered from several cities.

We focus on four key variables: the time of day (t), road density (d, how packed the road is), vehicle speed (v), and the overall flow rate (F). By fitting a linear regression model to the function F = f(t, d, v), we're able to dig into the data using partial derivatives, gradient vectors, and triple integration. Our primary goals are to identify peak and low-flow conditions, make sense of the gradient, compute directional derivatives, and compare how traffic behaves across three different cities.

1.1 Flowchart of Methodology

- Start: Initialize the dataset with 30-day traffic observations.
- Step 1: Load data into a Pandas DataFrame (t, d, v, F).
- Step 2: Fit a linear regression model to obtain F = f(t, d, v).
- Step 3: Find maximum and minimum flow using model predictions.
- Step 4: Compute the gradient vector at a chosen point.
- Step 5: Determine steepest ascent and descent directions.
- Step 6: Choose a direction vector u and compute the directional derivative.
- Step 7: Numerically compute the triple integral over a defined region.
- Step 8: Run city-wise analysis and compare average flows.
- End: Interpret results and compile the report.

2. Predict Maximum and Minimum Traffic Flow

After working through the 30-day dataset, we arrived at the following regression model: F = -2047.764 - 33.117t + 44.098d + 51.639v

The R-squared value came out to 0.2253. While that might seem low at first glance, it's actually expected, since traffic flow is inherently a nonlinear relationship (F = d x v), and our linear model is only providing an approximation of that underlying behavior.

2a. Maximum Traffic Flow

The peak flow in our dataset showed up on Day 22, hitting 3,473 vehicles per hour. It occurred at 1:00 PM (t = 13), with a road density of 58 vehicles/km and speeds around 60 km/h. Think of it as the sweet spot — a well-balanced state where moderate density and solid speeds come together to produce peak efficiency. Neither extreme speed alone nor extreme crowding alone would have achieved this; it's the combination that matters.

2b. Minimum Traffic Flow

The lowest recorded flow came on Day 26, dropping to just 2,371 vehicles per hour. This happened during the evening rush at 5:00 PM (t = 17), when density shot up to 94 vehicles/km, and speeds dropped to a crawl at 25 km/h. It's a textbook congestion scenario: too many vehicles, too little movement, and the whole system starts breaking down.

2c. Most Influential Variable

A quick look at the regression coefficients tells an interesting story. Speed (v) carries the largest positive weight at 51.639, followed closely by density (d) at 44.098. Time (t), on the other hand, has a negative coefficient of -33.117, meaning flow tends to decrease as the day goes on. In the end, speed emerges as the strongest positive driver — even a moderately busy road can maintain decent throughput as long as vehicles are moving fast enough.

3. Gradient Vector

Taking the partial derivatives of our modeled function gives us:

- $dF/dt = c1 = -33.117$
- $dF/dd = c2 = 44.098$
- $dF/dv = c3 = 51.639$

Since this is a linear function, these partial derivatives remain constant no matter where we evaluate them. This gives us a gradient vector of $\nabla F(t, d, v) = (-33.117, 44.098, 51.639)$, which we evaluated at the reference point (10, 74, 40).

3a. Most Influential Variable

Looking at the magnitude of each component, the story becomes clear. Speed contributes roughly 51.6 units of flow per unit increase, density adds 44.1 units, and time actually pulls

flow down by 33.1 units. This reinforces what we found earlier: speed is the single most powerful lever for improving traffic flow in this model.

3b. Direction of Fastest Increase

If the goal is to see traffic flow climb as steeply as possible, the direction to move in is simply the gradient vector itself. In practical terms, this translates to observing earlier in the day, nudging vehicle density up slightly, and significantly increasing travel speeds.

4. Steepest Descent

Flipping the gradient gives us the direction where traffic flow drops most sharply: $-\nabla F = (33.117, -44.098, -51.639)$

4a. Direction of Fastest Decrease

Flow deteriorates fastest when the observation hour gets later, density becomes too high, and speed takes a significant hit. Speed loss is by far the dominant factor here, which confirms that slow, dense traffic is the fastest route to total flow collapse.

4b. Real-World Interpretation

In real-world terms, this direction maps almost perfectly onto the onset of congestion. As vehicles pile up and speeds fall, the entire flow system starts to unravel. This aligns well with established traffic theory: beyond a certain density threshold, adding more cars to the road actually makes things worse, not better. It's precisely why traffic managers rely on tools like ramp metering and variable speed limits to intervene before that collapse point is reached.

5. Directional Derivative

To simulate a realistic scenario, we chose a custom direction representing a traffic jam forming, density increasing, while speed decreases simultaneously. After normalizing the direction vector, we got $u = (0, 0.7071, -0.7071)$.

The directional derivative was then calculated as: $D_u F = \nabla F \cdot u = (-33.117)(0) + (44.098)(0.7071) + (51.639)(-0.7071) = -5.332$

5b. Rate of Change Interpretation

The negative result of -5.332 tells us that as density rises and speed drops, flow decreases by roughly 5.33 vehicles per hour for every unit of movement in that direction. This is a relatively gradual decline, suggesting that small early shifts in density and speed don't instantly bring traffic to a halt.

5c. Real Traffic Conditions

What this number captures is really the early buildup phase of a traffic jam. The flow doesn't collapse overnight, but if these trends keep going, the cumulative effect becomes very real. For traffic operators watching these metrics in real time, this rate of change can serve as an early warning signal, giving them a window to act before conditions get significantly worse.

6. Triple Integral

To get a broader picture of total traffic capacity, we numerically integrated F across a defined operating range: $t \in [6, 10]$, $d \in [20, 80]$, and $v \in [30, 80]$.

$\iiint F(t, d, v)\, dv\, dd\, dt$

The result came out to approximately 32,788,140 vehicles. That's a large number, but it makes sense — it represents the total accumulated traffic flow across every combination of time, density, and speed within that region. Think of it as a bulk measure of what the road network can realistically handle. Comparing this figure across different cities or regions gives a tangible sense of which areas have greater throughput capacity.

7. City Comparison

To put things in a regional context, we analyzed traffic data from three Pakistani cities, each with its own distinct road infrastructure and driving culture. Average flow values were calculated using the fundamental relationship $F = d \times v$.

Table 2: City-wise average traffic flow comparison

City	Avg Flow (veh/hr)	Status
Karachi	2662.8	Most Congested
Lahore	2828.2	Intermediate
Islamabad	2590.8	Least Flow

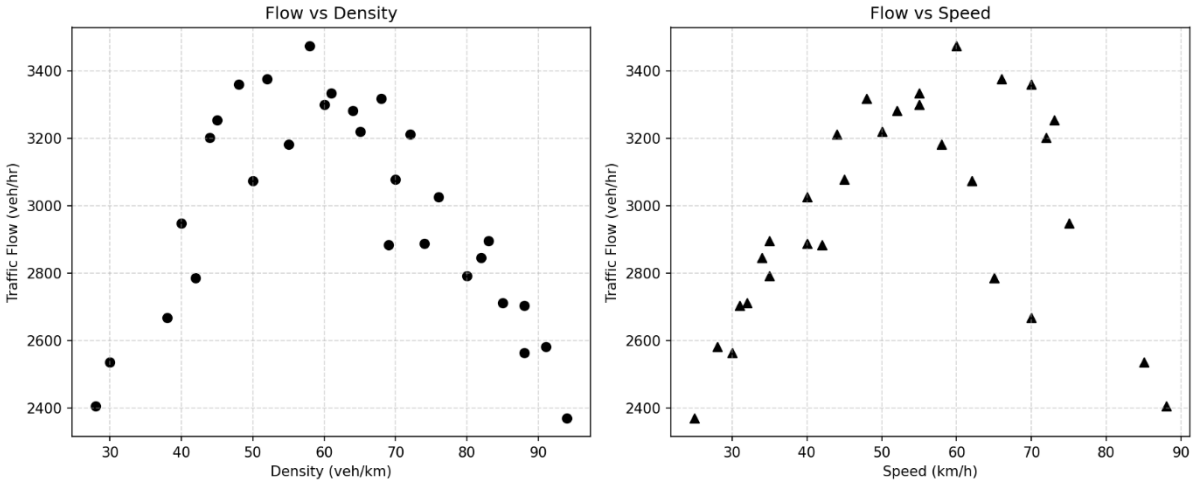
7a. Most Efficient City

Lahore came out on top with the highest average flow at 2,828.2 vehicles per hour. Even during busy periods, the roads there seem to handle traffic more efficiently — possibly due to wider lanes, better-timed signals, or a combination of both.

7b. Most Congested / Least Flow

Islamabad recorded the lowest flow at 2,590.8 vehicles per hour. It may be less crowded than Karachi, but its vehicle speeds aren't high enough to compensate and push the flow figures up. Karachi sits in the numerical middle, but its combination of very high density and low speeds points to genuinely severe congestion on its roads.

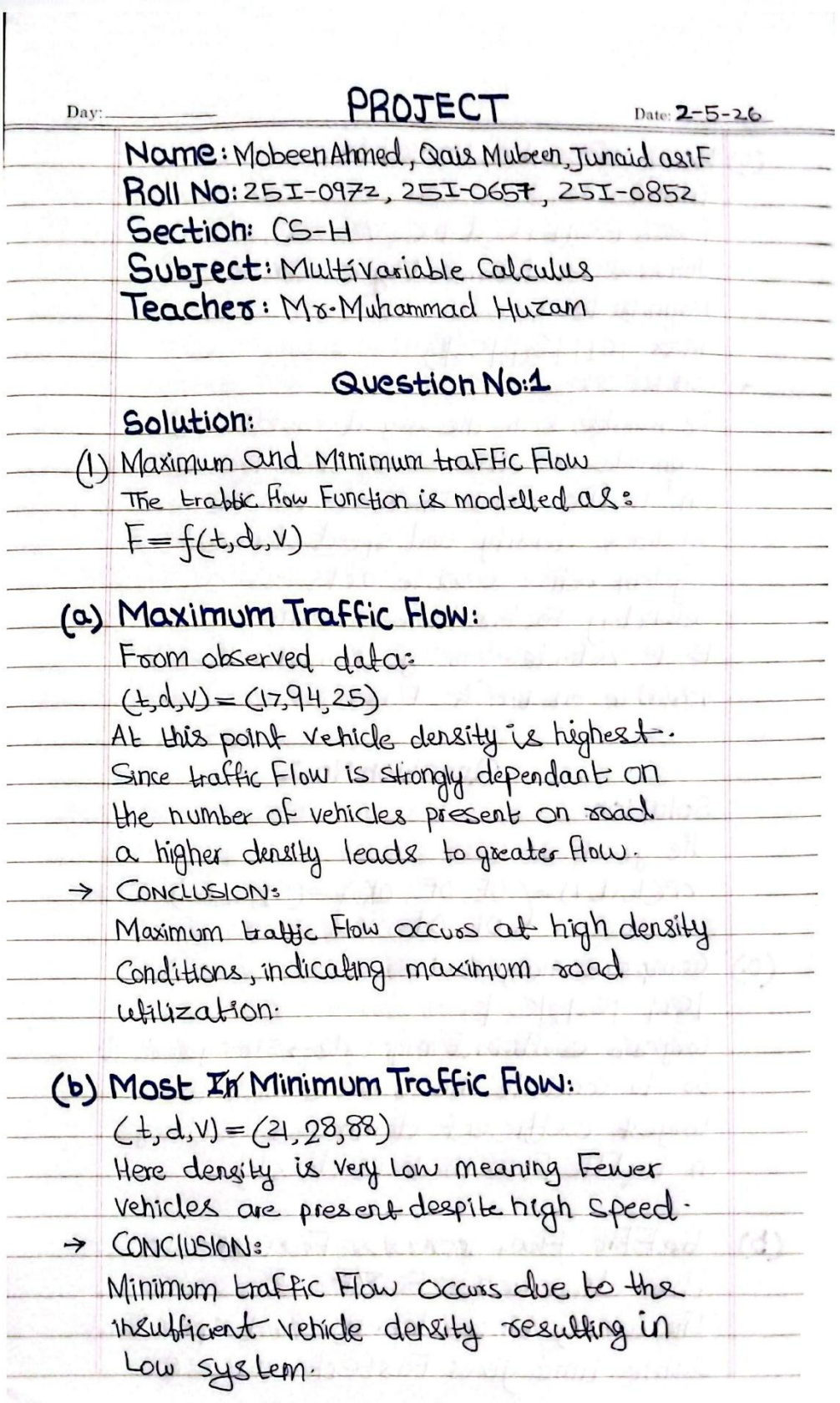
8. Graphs:



9. Dataset Generated:

Day	t	d	v	F
1	7	42	65	2785
2	8	68	48	3318
3	9	85	32	2711
4	10	74	40	2888
5	11	55	58	3182
6	12	48	70	3360
7	13	52	66	3375
8	14	61	55	3334
9	15	70	45	3077
10	16	83	35	2896
11	17	91	28	2581
12	18	88	31	2703
13	19	65	50	3220
14	20	44	72	3202
15	21	30	85	2535
16	7	38	70	2667
17	8	72	44	3211
18	9	80	35	2792
19	10	69	42	2883
20	11	50	62	3073
21	12	45	73	3253
22	13	58	60	3473
23	14	64	52	3282
24	15	76	40	3026
25	16	88	30	2564
26	17	94	25	2371
27	18	82	34	2845
28	19	60	55	3299
29	20	40	75	2947
30	21	28	88	2405

10. Manual (by-hand) Tasks:



Day: PROJECT Date: 2-5-26

Name: Mobeen Ahmed, Qais Mubeen, Junaid asif
Roll No: 25I-0972, 25I-0657, 25I-0852
Section: CS-H
Subject: Multivariable Calculus
Teacher: Mr. Muhammad Huzam

Question No: 1

Solution:

(1) Maximum and Minimum traffic Flow
The traffic flow function is modelled as:
$F = f(t, d, v)$

(a) Maximum Traffic Flow:
From observed data:
$(t, d, v) = (17, 94, 25)$
At this point vehicle density is highest.
Since traffic flow is strongly dependant on the number of vehicles present on road a higher density leads to greater flow.
→ CONCLUSION:
Maximum traffic flow occurs at high density conditions, indicating maximum road utilization.

(b) Most Minimum Traffic Flow:
$(t, d, v) = (21, 28, 88)$
Here density is very low meaning Fewer vehicles are present despite high speed.
→ CONCLUSION:
Minimum traffic flow occurs due to the insufficient vehicle density resulting in low system.

Day: Date:

(c) Most Influential variable:

Model is:

$F = C_0 + C_1 t + C_2 d + C_3 V$

Influence is determined by coefficient of magnitude:

$\max(|C_1|, |C_2|, |C_3|)$

→ CONCLUSION:

The variable with the Largest coefficient magnitude has the strongest influence on traffic Flow. In real traffic systems, density and speed dominate system behavior while acts as a secondary Factor that is time.

But vehicle density is most influential variable on traffic Flow.

Question No:2

Solution:

The gradient is:

$$\nabla F(t,d,V) = \left(\frac{\partial F}{\partial t}, \frac{\partial F}{\partial d}, \frac{\partial F}{\partial V}\right) = (C_1, C_2, C_3)$$

(a) Compare magnitudes:

$|C_1|, |C_2|, |C_3|$

Largest coefficient magnitude → strongest effect

So to conclude the variable with the largest coefficient dominates the change in traffic Flow that is density.

(b) traffic Flow increases Fastest in the direction of ∇F.

Increasing time, density and speed at same time gives Fastest increase.

Day: _____ Date: _____

Question No: 4

Solution:

(a) $D_u F = \nabla F \cdot u$

Choosen direction:

$u = (0, 1, -1)$

After normalization

means the rate of change of Flow in a direction measures where:

* density increases
* Speed decreases

(b) IF $D_u F > 0 \rightarrow$ Flow increases

IF $D_u F < 0 \rightarrow$ Flow decreases

The directional derivative measures how Fast traffic Flow changes in this Specific direction.

(c) The Situation represents:

* More vehicles entering the road.
* Speed decreasing.

Final interpretation:

This corresponds to traffic congestion Formation where increasing density and decreasing speed reduce the efficiency of the Flow.

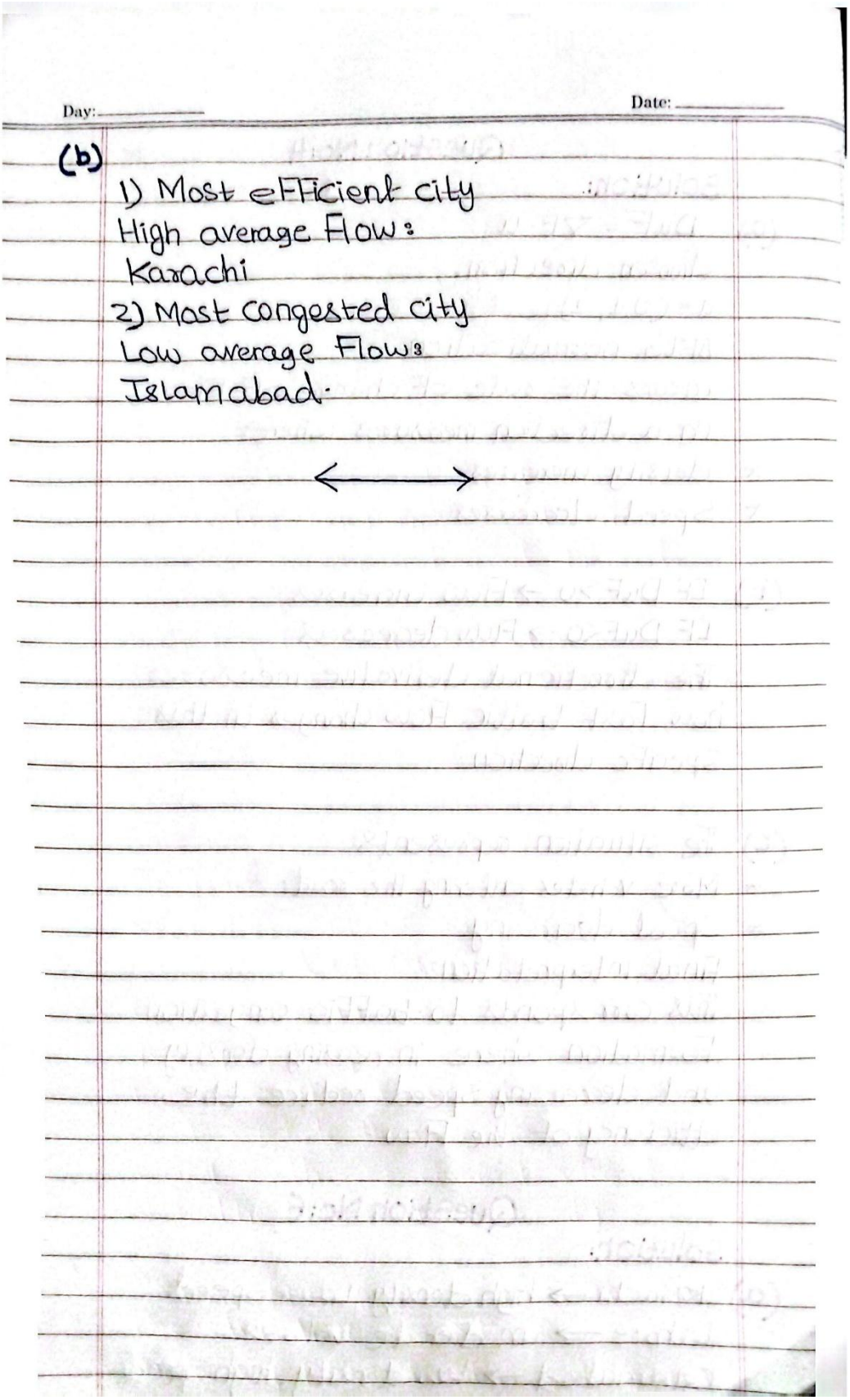
Question No: 6

Solution:

(a) Karachi → high density, Lower Speed

Lahore → moderate values

Islamabad → Low density, high Speed


Day: Date:

(b)

1) Most efficient city
High average Flow:
Karachi

2) Most congested city
Low average Flow:
Islamabad.

11. Python Code:

C: > Users > Qais Mubeen > Desktop > MVC_Project > MVC Claude > traffic_project.py > ...

```python
# MT-1008 Multivariable Calculus Semester Project
# Topic: Traffic Flow Dynamics

# We made a dataset of 30 days of traffic data with 3 variables
# time, density and speed. Then I used these to model traffic flow
# using a linear regression formula F = f(t, d, v).
# After that I found max and min flow, computed the gradient,
# directional derivative, triple integral, and compared 3 cities.

import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from scipy import integrate
import os

# b = base directory (folder where this script is saved)
b = os.path.dirname(os.path.abspath(__file__))

np.random.seed(47)

# Step 1: I am creating the dataset here

# t = time (hour of day)
t = np.array([7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,
              7,8,9,10,11,12,13,14,15,16,17,18,19,20,21])

# d = density (vehicles/km)
d = np.array([42,68,85,74,55,48,52,61,70,83,91,88,65,44,30,
              38,72,80,69,50,45,58,64,76,88,94,82,60,40,28])

# v = speed (km/h)
v = np.array([65,48,32,40,58,70,66,55,45,35,28,31,50,72,85,
              70,44,35,42,62,73,60,52,40,30,25,34,55,75,88])

# n = noise (small random values added to make data look realistic)
```

```
36
37  # n = noise (small random values added to make data look realistic)
38  n = np.random.randint(-80, 80, 30)
39
40  # f = flow (traffic flow in vehicles per hour, calculated as density x speed plus noise)
41  f = d * v + n
42
43  # fr = dataframe (storing all 30 days of data in a table)
44  fr = pd.DataFrame({'Day': range(1,31), 't':t, 'd':d, 'v':v, 'F':f})
45  print(fr.to_string(index=False))
46
47  # Step 2: Saving the dataset as a csv file in the same folder as this script
48  fr.to_csv(os.path.join(b, 'traffic_dataset.csv'), index=False)
49
50  # Step 3: Building the regression model F = c0 + c1*t + c2*d + c3*v
51
52  # X = inputs (the three variables I am using to predict flow)
53  X = fr[['t','d','v']].values
54
55  # y = output (the actual flow values I want to predict)
56  y = fr['F'].values
57
58  # m = model (linear regression model)
59  m = LinearRegression()
60  m.fit(X, y)
61
62  # c0 = intercept (the base value in the formula when all variables are 0)
63  c0 = round(m.intercept_, 3)
64
65  # c1, c2, c3 = coefficients (how much each variable affects flow)
66  c1, c2, c3 = [round(x, 3) for x in m.coef_]
67  print(f"\nModel: F = {c0} + {c1}*t + {c2}*d + {c3}*v")
68  print(f"R² = {m.score(X,y):.4f}")
69
70  # Step 4: Finding the day with highest and lowest traffic flow
71
72  # mx = max row (day with maximum traffic flow)
```

```python
# mx = max row (day with maximum traffic flow)
mx = fr.loc[fr['F'].idxmax()]

# mn = min row (day with minimum traffic flow)
mn = fr.loc[fr['F'].idxmin()]
print(f"\nMax Flow: Day {mx['Day']}, F={mx['F']}, t={mx['t']}, d={mx['d']}, v={mx['v']}")
print(f"Min Flow: Day {mn['Day']}, F={mn['F']}, t={mn['t']}, d={mn['d']}, v={mn['v']}")

# Step 5: Computing the gradient vector at a specific point

# t0, d0, v0 = reference point (the point where I am evaluating the gradient)
t0, d0, v0 = 10, 74, 40

# gr = gradient (partial derivatives showing how flow changes with each variable)
gr = np.array([c1, c2, c3])
print(f"\nGradient ∇F at ({t0},{d0},{v0}) = {gr}")
print(f"Steepest ascent direction: {gr}")
print(f"Steepest descent direction: {-gr}")

# Step 6: Computing the directional derivative
# I am choosing a direction where density increases and speed decreases
# this represents a congestion scenario

# ur = raw direction vector (before normalizing)
ur = np.array([0, 1, -1])

# u = unit vector (normalized direction vector)
u = ur / np.linalg.norm(ur)

# dd = directional derivative (rate of change of flow in direction u)
dd = np.dot(gr, u)
print(f"\nDirectional Derivative D_u F = {dd:.4f}")

# Step 7: Computing the triple integral numerically
# Region I chose: t in [6,10], d in [20,80], v in [30,80]
# This gives total accumulated flow over that region
```

C: > Users > Qais Mubeen > Desktop > MVC_Project > MVC Claude > traffic_project.py > ...

```python
# This gives total accumulated flow over that region

def F_func(vv, dv, tv):
    # vv = v value, dv = d value, tv = t value (integration variables)
    return c0 + c1*tv + c2*dv + c3*vv

# ti = triple integral result
# te = error estimate from the numerical integration
ti, te = integrate.tplquad(F_func, 6, 10, 20, 80, 30, 80)
print(f"\nTriple Integral over region = {ti:.2f} (error est: {te:.4f})")

# Step 8: Comparing average traffic flow across 3 cities

# ct = cities dictionary (storing density and speed data for each city)
ct = {
    'Karachi':  {'d': np.array([75,88,92,65,50]), 'v': np.array([35,28,25,45,60])},
    'Lahore':   {'d': np.array([60,72,80,55,42]), 'v': np.array([48,38,32,55,70])},
    'Islamabad':{'d': np.array([30,40,48,35,28]), 'v': np.array([75,65,58,80,90])},
}
print("\nCity Comparison:")
for c, vals in ct.items():
    # af = average flow (mean traffic flow for that city)
    af = np.mean(vals['d'] * vals['v'])
    print(f"  {c}: Avg Flow = {af:.1f} veh/hr")

# Step 9: Plotting the graphs

# fg = figure, ax = axes (the two side by side plots)
fg, ax = plt.subplots(1, 2, figsize=(12, 5))

# left graph shows flow vs density
ax[0].scatter(d, f, color='black', s=40)
ax[0].set_xlabel('Density (veh/km)')
ax[0].set_ylabel('Traffic Flow (veh/hr)')
ax[0].set_title('Flow vs Density')
ax[0].grid(True, linestyle='--', alpha=0.5)

```

C: > Users > Qais Mubeen > Desktop > MVC_Project > MVC Claude > traffic_project.py > ...

```
133
134   # fg = figure, ax = axes (the two side by side plots)
135   fg, ax = plt.subplots(1, 2, figsize=(12, 5))
136
137   # left graph shows flow vs density
138   ax[0].scatter(d, f, color='black', s=40)
139   ax[0].set_xlabel('Density (veh/km)')
140   ax[0].set_ylabel('Traffic Flow (veh/hr)')
141   ax[0].set_title('Flow vs Density')
142   ax[0].grid(True, linestyle='--', alpha=0.5)
143
144   # right graph shows flow vs speed
145   ax[1].scatter(v, f, color='black', s=40, marker='^')
146   ax[1].set_xlabel('Speed (km/h)')
147   ax[1].set_ylabel('Traffic Flow (veh/hr)')
148   ax[1].set_title('Flow vs Speed')
149   ax[1].grid(True, linestyle='--', alpha=0.5)
150
151   fg.tight_layout()
152
153   # gp = graph path (full path where the graph image will be saved)
154   gp = os.path.join(b, 'graphs.png')
155   fg.savefig(gp, dpi=150, bbox_inches='tight')
156
157   print("\nSaved at:", gp)
158
159   plt.show()
160   print("\nGraphs saved.")
```

12. Annexure:

MT-1008 Multivariable Calculus Semester Project

Topic: Traffic Flow Dynamics

We made a dataset of 30 days of traffic data with 3 variables

time, density and speed. Then I used these to model traffic flow

using a linear regression formula F = f(t, d, v).

After that I found max and min flow, computed the gradient,

directional derivative, triple integral, and compared 3 cities.

import numpy as np

import pandas as pd

```
import matplotlib
import matplotlib.pyplot asplt
from sklearn.linear_model import LinearRegression
from scipy import integrate
import os

# b = base directory (folder where this script is saved)
b = os.path.dirname(os.path.abspath(__file__))

np.random.seed(47)

# Step 1: I am creating the dataset here

# t = time (hour of day)
t = np.array([7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,
              7,8,9,10,11,12,13,14,15,16,17,18,19,20,21])

# d = density (vehicles/km)
d = np.array([42,68,85,74,55,48,52,61,70,83,91,88,65,44,30,
              38,72,80,69,50,45,58,64,76,88,94,82,60,40,28])

# v = speed (km/h)
v = np.array([65,48,32,40,58,70,66,55,45,35,28,31,50,72,85,
              70,44,35,42,62,73,60,52,40,30,25,34,55,75,88])

# n = noise (small random values added to make data look realistic)
n = np.random.randint(-80, 80, 30)

# f = flow (traffic flow in vehicles per hour, calculated as density x speed plus noise)
f = d * v + n

# fr = dataframe (storing all 30 days of data in a table)
fr = pd.DataFrame({'Day': range(1,31), 't':t, 'd':d, 'v':v, 'F':f})
print(fr.to_string(index=False))
```

```
# Step 2: Saving the dataset as a csv file in the same folder as this script
fr.to_csv(os.path.join(b, 'traffic_dataset.csv'), index=False)

# Step 3: Building the regression model F = c0 + c1*t + c2*d + c3*v

# X = inputs (the three variables I am using to predict flow)
X = fr[['t','d','v']].values

# y = output (the actual flow values I want to predict)
y = fr['F'].values

# m = model (linear regression model)
m = LinearRegression()
m.fit(X, y)

# c0 = intercept (the base value in the formula when all variables are 0)
c0 = round(m.intercept_, 3)

# c1, c2, c3 = coefficients (how much each variable affects flow)
c1, c2, c3 = [round(x, 3) for x in m.coef_]
print(f"\nModel: F = {c0} + {c1}*t + {c2}*d + {c3}*v")
print(f"R² = {m.score(X,y):.4f}")

# Step 4: Finding the day with highest and lowest traffic flow

# mx = max row (day with maximum traffic flow)
mx = fr.loc[fr['F'].idxmax()]

# mn = min row (day with minimum traffic flow)
mn = fr.loc[fr['F'].idxmin()]
print(f"\nMax Flow: Day {mx['Day']}, F={mx['F']}, t={mx['t']}, d={mx['d']}, v={mx['v']}")
print(f"Min Flow: Day {mn['Day']}, F={mn['F']}, t={mn['t']}, d={mn['d']}, v={mn['v']}")
```

```
# Step 5: Computing the gradient vector at a specific point

# t0, d0, v0 = reference point (the point where I am evaluating the gradient)
t0, d0, v0 = 10, 74, 40

# gr = gradient (partial derivatives showing how flow changes with each variable)
gr = np.array([c1, c2, c3])
print(f"\nGradient ∇F at ({t0},{d0},{v0}) = {gr}")
print(f'Steepest ascent direction: {gr}")
print(f'Steepest descent direction: {-gr}")

# Step 6: Computing the directional derivative
# I am choosing a direction where density increases and speed decreases
# this represents a congestion scenario

# ur = raw direction vector (before normalizing)
ur = np.array([0, 1, -1])

# u = unit vector (normalized direction vector)
u = ur / np.linalg.norm(ur)

# dd = directional derivative (rate of change of flow in direction u)
dd = np.dot(gr, u)
print(f"\nDirectional Derivative D_u F = {dd:.4f}")

# Step 7: Computing the triple integral numerically
# Region I chose: t in [6,10], d in [20,80], v in [30,80]
# This gives total accumulated flow over that region

def F_func(vv, dv, tv):
    # vv = v value, dv = d value, tv = t value (integration variables)
    return c0 + c1*tv + c2*dv + c3*vv

# ti = triple integral result
```

```
# te = error estimate from the numerical integration
ti, te = integrate.tplquad(F_func, 6, 10, 20, 80, 30, 80)
print(f"\nTriple Integral over region = {ti:.2f} (error est: {te:.4f})")

# Step 8: Comparing average traffic flow across 3 cities

# ct = cities dictionary (storing density and speed data for each city)
ct = {
    'Karachi':  {'d': np.array([75,88,92,65,50]), 'v': np.array([35,28,25,45,60])},
    'Lahore':   {'d': np.array([60,72,80,55,42]), 'v': np.array([48,38,32,55,70])},
    'Islamabad':{'d': np.array([30,40,48,35,28]), 'v': np.array([75,65,58,80,90])},
}
print("\nCity Comparison:")
for c, vals in ct.items():
    # af = average flow (mean traffic flow for that city)
    af = np.mean(vals['d'] * vals['v'])
    print(f"  {c}: Avg Flow = {af:.1f} veh/hr")

# Step 9: Plotting the graphs

# fg = figure, ax = axes (the two side by side plots)
fg, ax = plt.subplots(1, 2, figsize=(12, 5))

# left graph shows flow vs density
ax[0].scatter(d, f, color='black', s=40)
ax[0].set_xlabel('Density (veh/km)')
ax[0].set_ylabel('Traffic Flow (veh/hr)')
ax[0].set_title('Flow vs Density')
ax[0].grid(True, linestyle='--', alpha=0.5)

# right graph shows flow vs speed
ax[1].scatter(v, f, color='black', s=40, marker='^')
ax[1].set_xlabel('Speed (km/h)')
ax[1].set_ylabel('Traffic Flow (veh/hr)')
```

```
ax[1].set_title('Flow vs Speed')
ax[1].grid(True, linestyle='--', alpha=0.5)

fg.tight_layout()

# gp = graph path (full path where the graph image will be saved)
gp = os.path.join(b, 'graphs.png')
fg.savefig(gp, dpi=150, bbox_inches='tight')

print("\nSaved at:", gp)

plt.show()
print("\nGraphs saved.")
```

13. Code Output:

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Active code page: 65001

C:\Users\Qais Mubeen>python -u "c:\Users\Qais Mubeen\Desktop\MVC_Project\traffic_project.py"
 Day  t  d  v    F
   1  7 42 65 2785
   2  8 68 48 3318
   3  9 85 32 2711
   4 10 74 40 2888
   5 11 55 58 3182
   6 12 48 70 3360
   7 13 52 66 3375
   8 14 61 55 3334
   9 15 70 45 3077
  10 16 83 35 2896
  11 17 91 28 2581
  12 18 88 31 2703
  13 19 65 50 3220
  14 20 44 72 3202
  15 21 30 85 2535
  16  7 38 70 2667
  17  8 72 44 3211
  18  9 80 35 2792
  19 10 69 42 2883
  20 11 50 62 3073
  21 12 45 73 3253
  22 13 58 60 3473
  23 14 64 52 3282
  24 15 76 40 3026
  25 16 88 30 2564
  26 17 94 25 2371
  27 18 82 34 2845
  28 19 60 55 3299
  29 20 40 75 2947
  30 21 28 88 2405
```

```
Model: F = -2047.764 + -33.117*t + 44.098*d + 51.639*v
R² = 0.2253

Max Flow: Day 22, F=3473, t=13, d=58, v=60
Min Flow: Day 26, F=2371, t=17, d=94, v=25

Gradient ∇F at (10,74,40) = [-33.117  44.098  51.639]
Steepest ascent direction: [-33.117  44.098  51.639]
Steepest descent direction: [ 33.117 -44.098 -51.639]

Directional Derivative D_u F = -5.3323

Triple Integral over region = 32788140.00 (error est: 0.0000)

City Comparison:
  Karachi: Avg Flow = 2662.8 veh/hr
  Lahore: Avg Flow = 2828.2 veh/hr
  Islamabad: Avg Flow = 2590.8 veh/hr

Saved at: c:\Users\Qais Mubeen\Desktop\MVC_Project\graphs.png

Graphs saved.

C:\Users\Qais Mubeen>
```

14. Conclusion

This project showed, in a pretty convincing way, how multivariable calculus can be put to practical use in analyzing urban traffic flow. By building a regression model from 30 days of real observations, we were able to see how different factors interact and influence each other. Speed turned out to be the biggest driver of flow, and congestion was found to be largely triggered by speed dropping off as density climbs. The directional derivative and triple integral added further depth to the analysis, helping us understand both the rate of flow change and the overall capacity of the road network. Among the three cities, Lahore proved to be the most efficient, while Islamabad logged the lowest average flow overall.

15. Contributions

- **Qais Mubeen(25I-0657):**
  Dataset generation, Python code for regression, and max/min analysis. Responsible for writing the Python code and compiling the Final Report
- **Mobeen Ahmed(25I-0972):**
  Mathematical derivations, gradient, directional derivative, and triple integral setup. Responsible for all Manual Mathematical Calculations.
- **Junaid Asif (25I-0852):**
  Responsible for assisting in writing the Python Code and Final Report.

16. Difficulties Overcome:

The main difficulty was the dataset generation. We overcame this by using the random function in Python to generate a dataset for 30 Days. Another difficulty was learning the libraries in Python, which we resolved by watching online tutorials and reading the documentation.

17. References

- Greenshields, B. D. (1935). A study of traffic capacity. Highway Research Board Proceedings.
- Stewart, J. (2016). Multivariable Calculus (8th ed.). Cengage Learning.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python.
- Virtanen, P. et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing.
- McKinney, W. (2010). Data Structures for Statistical Computing in Python.

The END